

SSD_Environmental Construction

1 Overview

This document is mainly for the SSD20X platform and consists of two parts:

1. Setting up of Linux Compilation Environment
 2. SDK compilation method
-

2. Build the compilation environment

2.1. Overview of the debugging environment

Usually we develop and debug in the form of cross-compilation, that is, in the form of "host machine + target machine". For the connection between the host computer and the target computer, we generally use serial port connection or network connection, as shown in the following figure:

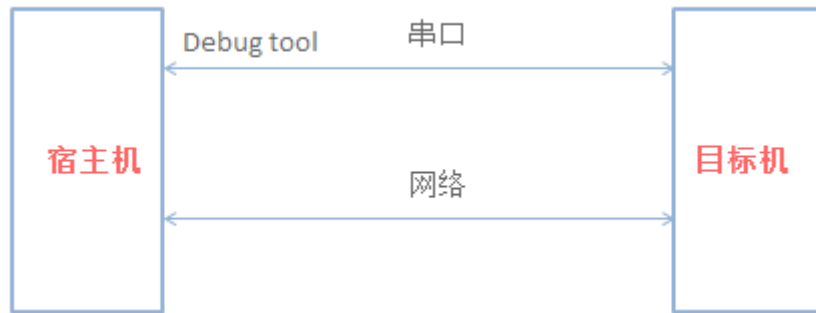


Figure 2-1 Debug connection mode

1. The serial port debugging is only used to capture the log, and the serial port device can be directly connected to the RX/TX of the uart0 (the default baud rate of Cosole is 115200)
2. If you need to debug/burn/read and write register operations for tools that use SSD platforms (eg: Flash_Tool), you need to [purchase a dedicated Debug Tool](https://item.taobao.com/item.htm?id=15437784110) [https://item.taobao.com/item.htm?id=15437784110]

2.2. Linux server

Ubuntu 16.04 is recommended.

The compilation of the SDK requires some lib&tools. The installation process is as follows.

2.3. Install ubuntu server

SSD201(64M DDR)

1. Download [1604 server version](https://www.ubuntu.org.cn/download/server) [https://www.ubuntu.org.cn/download/server]
2. Install VMware Workstation
3. Select the 1604 iso, mount it and install it
4. Simply click to complete the simple installation, the system will automatically install
5. Install VMtools for copying, share the windows directory to ubuntu, [click to view the reference](#)

[<https://www.cnblogs.com/huangjianxin/p/6343881.html>] .

The ubuntu installation was successful. Next, we will introduce the environment required for installation and development.

2.4. Install lib & tools required for compilation

Next, install the corresponding compilation tools, etc.:

1. Newly installed ubuntu, you must first set the root password, enter the system to set the root password: `# sudo passwd` , enter the password continuously, and the prompt can be successful.
2. When adding a user, you will be prompted for input when you create a virtual machine `user account` , but if you want to use `beal.wu` a similar account, you cannot enter it when you create ubuntu (it will prompt a format error), and the system will prompt that the account is illegal, but because you need to use `beal.wu` , then you need to re-create the user after entering the system `# sudo adduser beal.wu--force` to create a user. At this time, you will be prompted to enter some account information, just press Enter.
3. At this point the account has been established, then you need to install the relevant tools and compilation environment, execute

```
# sudo apt - get  
  
install  libncurses5 - dev  
  
libncursesw5 - dev
```

At this time, you will be prompted that `beal.wu` cannot execute sudoers, you need to make the following modifications:

```
sudo chmod u+w /etc/sudoers  
vi /etc/sudoers;  
refer to root ALL=(ALL:ALL) ALL,  
add a line beal.wu ALL=(ALL:ALL) ALL
```

Then execute the sudo operation to install it. It is recommended `#sudo apt-get update` to update the source after the installation is complete.

4. Install samba for sharing linux files to windows

```
# sudo apt-get install samba samba-common
# sudo smbpasswd -a beal.wu (add samba account for windows to
access the linuxsamba directory)
Modify /etc/samba/smb.conf
[beal.wu]
path=/home/beal .wu
public=yes
writable=yes
valid users=beal.wu
available=yes
browseable=yes
guest ok = yes
```

In this way, samba has been configured, restart the samba service, and test accessing samba from windows: \\192.168.1.11 (your ubuntu ip)

5. Install ssh for login debugging

```
sudo apt-get install openssh-server;
```

Ready to use after installation

6. You need to install some tools for SDK compilation, otherwise the compilation will fail

```
# sudo apt-get install libc6-dev-i386
# sudo apt-get install lib32z1 lib32ncurses5
# sudo apt-get install libuuid1:i386
# sudo apt-get install cmake
# sudo apt-get install libncurses5-dev libncursesw5-dev
# sudo apt install bc
# sudo apt-get install xz-utils
# sudo apt-get install automake
# sudo apt-get install libtool
# sudo apt-get install libevdev-dev
# sudo apt-get install pkg-config
```

如上这些需要安装的tool && lib是必须的，因为编译过程中会用到相关内容。这边就不一一说明是哪个错误，可以尝试先不安装编译来查看相关错误，为了方便，在编译之前一次性安装即可。

7. 如果默认sh不是bash，需要将 sh 改成 bash：

```
Sudo rm /bin/sh
```

```
Sudo ln -s /bin/bash /bin/sh
```

8. 安装toolchain

- 拷贝 `gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi` 到 `/opt/` 目录自行定义
- 解压toolchain
- 将toolchain设置到环境变量中去，省得每次需要手动export；
- 在 `/etc/profile` 添加：`export PATH=/opt/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi/bin:$PATH`

9. 执行编译

2.5. 安装交叉编译工具

我们提供编译工具链 `gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi` 用于编译glibc版本的。

Glibc : 2.28

```
beal.wu@sigmastar:~/i2m/sdk$ /opt/ToolChain/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi/bin/arm-linux-gnueabi-gc
Using built-in specs.
COLLECT_GCC=/opt/ToolChain/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi/bin/arm-linux-gnueabi-gcc
COLLECT_LTO_WRAPPER=/opt/ToolChain/gcc-arm-8.2-2018.08-x86_64-arm-linux-gnueabi/bin/../libexec/gcc/arm-linux-gnueabi
Target: arm-linux-gnueabi
Configured with: /tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/src/gcc/config
ith-sysroot=/arm-linux-gnueabi/libc --with-build-sysroot=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64
ueabi/install/arm-linux-gnueabi/libc --enable-gnu-indirect-function --enable-shared --disable-libssp --disab
ing=release --enable-languages=c,c++,fortran --with-gmp=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--a
bihf/host-tools --with-mpfr=/tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/bui
tmp/dgboter/bbs/bc-b1-2-11--rhe6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/build-arm-linux-gnueabi/host-t
he6x86_64/buildbot/rhe6x86_64--arm-linux-gnueabi/build/build-arm-linux-gnueabi/host-tools --with-arch=armv7-a --wi
7-a --with-pkgversion='GNU Toolchain for the A-profile Architecture 8.2-2018-08 (arm-rel-8.23)'
Thread model: posix
gcc version 8.2.1 20180802 (GNU Toolchain for the A-profile Architecture 8.2-2018-08 (arm-rel-8.23))
beal.wu@sigmastar:~/i2m/sdk$
```

图2-2

3. 获取SDK

3.1. SDK来源

请咨询FAE，从FTP中获取对应SDK，完整的SDK code应包含如下部分：

文件名	文件大小	文件类型
boot-TAKOYAKI_DLC00V030.tar.gz	15,424,920	WinRAR 压缩...
kernel-TAKOYAKI_DLC00V030.tar.gz	148,806,209	WinRAR 压缩...
project-TAKOYAKI_DLC00V030.tar.gz	865,329,391	WinRAR 压缩...
sdk-TAKOYAKI_DLC00V030.tar.gz	364,453,402	WinRAR 压缩...

boot：Uboot源码

kernel：kernel源码，版本：4.9.84

project：编译制作image的部分，包含非开源部分的lib/ko, 以及对外api头文件参考

sdk：测试Demo/应用打包框架部分

3.2. 解压SDK

```
tar zxvf project-TAKOYAKI_DLC00V030.tar.gz
tar zxvf boot-TAKOYAKI_DLC00V030.tar.gz
tar zxvf kernel-TAKOYAKI_DLC00V030.tar.gz
tar zxvf sdk-TAKOYAKI_DLC00V030.tar.gz
```

4. 编译SDK

SSD20x Relase的SDK可以支持Nor Flash和Nand Flash两种，只需要根据实际情况区分配置编译即可。

4.1. 编译链指定

配置好编译链：

```
declare -x ARCH="arm"
declare -x CROSS_COMPILE="arm-linux-gnueabihf-"
```

4.2. boot编译

1. 进入boot根目录
2. 根据Flash选择config

Nand Flash	Nor Flash
make infinity2m_spinand_defconfig; make menuconfig;	make infinity2m_defconfig; make menuconfig;

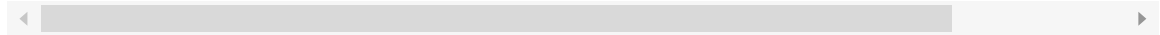
3. 执行 `make clean;make -j8` （根据服务器配置，可以支持多线程编译）
4. 生成image，如需要打包需要release到project对应目录

Nand Flash	Nor Flash
u-boot_spinand.xz.img.bin	u-boot.xz.img.bin
需要release到project目录： cp u-boot_spinand.xz.img.bin project\board\i2m\boot\spinand\uboot	需要release到project目录： cp u-boot.xz.img.bin project\board\i2m\boot\nor\uboot

4.3. kernel编译

1. 进入kernel根目录
2. 根据Flash选择config

Nand Flash	Nor Flash
make infinity2m_spinand_ssc011a_s01a_minigui_defconfig; make menuconfig;	make infinity2m_ssc011a_s01a_minigui_d ; menuconfig;



3. 执行 `make clean;make -j8` (根据服务器配置 , 可以支持多线程编译)
4. 生成image , 如需要打包需要Relase到project对应目录

Nand Flash	Nor Flash
arch/arm/boot/ulmage.xz	arch/arm/boot/ulmage.xz
需要release到project目录 : project\release\nvr\i2m\011A\glibc\8.2.1\bin\kernel\spinand	需要release到project目录 : project\release\nvr\i2m\011



4.4. project工程编译

1. 进入project根目录
2. 根据Flash_Type以及平台DDR差异选择config

Nand Flash	
SSD201(64M DDR)	SSD202(128M DDR)
./setup_config.sh ./configs/nvr/i2m/8.2.1/spinand.glibc.011a.64	./setup_config.sh ./configs/nvr/i2m/8.2.1/spinand.glibc.011a



Nor Flash SSD201(64M DDR)

```
./setup_config.sh ./configs/nvr/i2m/8.2.1/nor.glibc-squashfs.011a.64
```

3. Release kernel相关资源 (非必须动作 , 只有当改动到kernel部分config的时候需要执行此步)

Nand Flash

```
./release.sh -k ${kernel_path} -b 011A -p nvr -f spinand -c i2m -l glibc -v 8.2.1
```

Nor Flash

```
./release.sh -k ${kernel_path} -b 011A -p nvr -f nor -c i2m -l glibc -v 8.2.1
```

4. 执行:make image (project工程不支持多线程编译)

5. Generate image, directory: project/image/output/images

After getting the generated image, you can burn it to the board by referring to the burning document/master making document.

5. Instructions for third-party IDEs

Flying development reference comake: <http://dev.comake.online/thread-90-1-1.html> [http://dev.comake.online/thread-90-1-1.html]